UNIVERSIDAD TECNOLÓGICA DE GUTIÉRREZ ZAMORA

PROGRAMA EDUCATIVO:

Ingeniería en Desarrollo y Gestión de Software

NOMBRE DE LA MATERIA:

Desarrollo Móvil Integral

NOMBRE DE LA TAREA:

Unidad II — Seguridad MASVS

Auditoría básica de seguridad con OWASP MASVS / MASTG
(aplicación propia: TotoNinin)

ELABORADO POR:

Héctor Zahid Juárez Simbrón

DOCENTE:

Lothar Yamir Hernández Gaona

1010 - “AEI”

SEPTIEMBRE – DICIEMBRE 2026

Índice

Alcance y controles **3**
Datos y autenticación **5**
Red, permisos y configuración **8**
Hallazgos y correcciones **10**
Verificación e informe **23**

Alcance y controles

Alcance de la auditoría

Esta auditoría revisó la seguridad de TotoNinin, una aplicación web hecha en PHP 8 y MySQL, sin framework, que sirve para gestionar eventos, usuarios, categorías y comentarios. Las pruebas se hicieron en un entorno local, con una base de datos de prueba y solo con datos ficticios: usuarios inventados con correos @totoninin.test y claves de prueba (por ejemplo admin@totoninin.test y user@totoninin.test). En todas las evidencias, las contraseñas, los hashes y las cookies de sesión aparecen ocultos.

Se trabajó con dos versiones del código. v1-original es el proyecto tal como venía en el ZIP (lo recibí el 29/10/2026) y v2-corregida es el mismo proyecto con las correcciones aplicadas.

Referencia utilizada

Usé OWASP MASVS (Mobile Application Security Verification Standard) en su versión 2, y la guía MASTG para saber cómo probar cada punto. MASVS agrupa los controles en ocho áreas: STORAGE (almacenamiento de datos), CRYPTO (criptografía), AUTH (autenticación y permisos), NETWORK (comunicación por la red), PLATFORM (relación con el sistema), CODE (calidad y seguridad del código), RESILIENCE (resistencia a la ingeniería inversa) y PRIVACY (privacidad).

Controles evaluados y controles que no aplican

MASVS está pensado para apps móviles, y TotoNinin es una aplicación web que corre en un servidor: el proyecto no trae ningún APK ni IPA. Por eso evalué los controles que tienen un equivalente en una aplicación web: STORAGE, CRYPTO, AUTH, CODE y PLATFORM (en PLATFORM, el control de acceso a las funciones de administrador y la validación de los archivos que se suben). También revisé

PRIVACY: la app pide solo los datos necesarios (nombre, correo, contraseña y comentarios) y no usa rastreadores de terceros. Su única observación es que no hay aviso de privacidad ni una forma de eliminar la propia cuenta, y la dejé como mejora para después.

No aplica RESILIENCE, que protege contra la ingeniería inversa en el dispositivo (descompilar o modificar una app ya instalada), porque en TotoNinin el código se queda en el servidor y nunca llega al usuario. Tampoco aplican los puntos de PLATFORM que son propios del celular: permisos del sistema, WebViews (páginas web dentro de una app) y enlaces profundos. NETWORK no se evaluó a fondo porque el proyecto no incluye la configuración del servidor ni un certificado HTTPS, y en el entorno local se usó HTTP; para producción se recomienda activar HTTPS.

Escala de riesgo

Para el nivel de riesgo le puse a cada hallazgo una probabilidad (qué tan fácil es aprovecharlo) y un impacto (qué tanto daño haría), de 1 a 3 cada uno, y los multipliqué. Un resultado de 1 a 3 es Bajo, de 4 a 6 Medio, de 7 a 8 Alto y 9 Crítico. Es la escala de la unidad, no una escala oficial de OWASP.

Datos y autenticación

Aquí se revisó cómo maneja la aplicación las credenciales y las sesiones activas. Se encontraron cuatro problemas.

VULN-01 — Inyección SQL

La inyección SQL pasa cuando el programa arma una consulta a la base de datos pegando directamente lo que escribió el usuario. Si alguien escribe código SQL en lugar de un correo, la base de datos lo ejecuta como si fuera parte de la consulta. Pertenece a MASVS-CODE y, en segundo plano, MASVS-AUTH. Está en el login (scripts/login.php), en el parámetro "mes" de scripts/load_events.php y scripts/obtener_eventos.php, en el parámetro "pagina" de lista_usuarios.php, lista_eventos.php y lista_categoria.php, y en los formularios de registro_usuario.php, editar_usuario.php, registro_categoria.php y editar_categoria.php (correo y descripción).

Para comprobarlo mandé un POST a scripts/login.php con x' OR '1'='1 tanto en el correo como en la contraseña. Como la condición siempre sale verdadera, la app me dejó entrar sin conocer ninguna contraseña. Ya dentro con cualquier usuario, abrí scripts/load_events.php?mes=0 UNION SELECT ID_us,Name_us,password_us,... FROM users -- y esa página, que solo debería listar los eventos del mes, me mostró los hashes de contraseña de todos los usuarios.

Le puse probabilidad 3 (alta) e impacto 3 (alto), así que el riesgo es Crítico (9 de 9). Cualquiera puede entrar sin contraseña y leer los datos de todos los usuarios, y es muy fácil de hacer.

VULN-02 — Control de acceso roto en las funciones de administrador

Control de acceso roto quiere decir que la aplicación no revisa si quien pide una acción tiene permiso para hacerla. Las pantallas y acciones de administrador estaban abiertas para cualquiera. Pertenece a MASVS-AUTH y, en segundo

plano, MASVS-PLATFORM. Afecta a scripts/lista_usuarios.php, eliminar_usuario.php, editar_usuario.php, registro_usuario.php, lista_categoria.php, eliminar_categoria.php, registro_categoria.php, editar_categoria.php, lista_eventos.php, eliminar_confirmar_evento.php, registro_evento.php, editar_evento.php y paginas/admin.php.

Sin iniciar sesión pedí scripts/lista_usuarios.php y me mostró la lista completa de usuarios con nombre y correo. Con un POST a scripts/eliminar_usuario.php?id=3, también sin sesión, el usuario se borró de la base de datos. Con otro POST a scripts/editar_usuario.php?id=2 le cambié la contraseña a otro usuario y después pude entrar a su cuenta con la clave que puse yo. Por último, entré con un usuario normal (no administrador) y el panel de administración se abrió igual, porque ningún script revisa el rol.

Le puse probabilidad 3 (alta) e impacto 3 (alto), así que el riesgo es Crítico (9 de 9). Cualquier persona, incluso sin cuenta, puede ver, borrar y modificar usuarios, y hasta quedarse con la cuenta de otro.

VULN-03 — Contraseñas guardadas con MD5

Una base de datos no debe guardar la contraseña tal cual, sino un hash, que es el resultado de revolverla con una fórmula que no se puede deshacer. MD5 es una fórmula vieja y muy rápida, y como se usó sin sal (un dato aleatorio que se mezcla con la contraseña), es fácil adivinar contraseñas probando palabras de un diccionario. Pertenece a MASVS-CRYPTO y, en segundo plano, MASVS-STORAGE. Está en scripts/registro.php y scripts/registro_usuario.php (alta de usuarios desde el panel de administración).

Registré un usuario desde scripts/registro_usuario.php y revisé la columna password_us en la base de datos. El valor era un hash MD5 de 32 caracteres hexadecimales y sin sal, que se puede descifrar en segundos si la base de datos llegara a filtrarse. El formulario público de registro (paginas/registro.php) ya usaba

password_hash() con bcrypt desde el original, así que el problema estaba solo en el alta que hace el administrador y en la clase Registro, que quedó de una versión anterior.

Le puse probabilidad 2 (media) e impacto 2 (medio), así que el riesgo es Medio (4 de 9). Solo importa si alguien llega a ver la base de datos, y el registro público ya usaba bcrypt.

VULN-06 — Manejo inseguro de la sesión

La sesión es lo que mantiene a un usuario dentro de la aplicación después de iniciar sesión, y se guarda en una cookie del navegador. Si no se cuida, esa cookie se puede robar o reutilizar. Pertenece a MASVS-AUTH. Está en scripts/login.php, paginas/login.php, scripts/cerrar_sesion.php, includes/header.php e includes/nav.php.

Al iniciar sesión, la cabecera Set-Cookie del PHPSESSID no trae HttpOnly ni SameSite, y el identificador de sesión es el mismo que tenía el visitante antes de entrar porque no se llama a session_regenerate_id(). Eso permite que un atacante le dé a la víctima un identificador que él ya conoce, por ejemplo con un enlace, y lo siga usando cuando la víctima inicie sesión (fijación de sesión). Otro problema es que el login responde distinto cuando el correo no existe y cuando la contraseña es incorrecta, así que se pueden ir probando correos para saber cuáles están registrados. Además, scripts/cerrar_sesion.php llama a session_destroy() pero nunca borra la cookie del navegador, por lo que el identificador sigue viajando en cada petición. Por último, el rol de administrador se decide comparando el correo con un valor fijo dentro de includes/nav.php, repetido en varios archivos, en lugar de calcularlo una sola vez en el servidor al autenticar.

Le puse probabilidad 2 (media) e impacto 2 (medio), así que el riesgo es Medio (4 de 9). Cada problema por separado necesita condiciones extra, como engañar a la víctima o tener acceso a su navegador.

Red, permisos y configuración

Aquí se revisaron los archivos de configuración, lo que el proyecto guarda en sus archivos y los permisos de lo que se sube al servidor. Se encontraron dos problemas.

VULN-05 — Credenciales escritas dentro del código

Dejar usuarios y contraseñas escritos dentro del código es un problema porque cualquiera que vea el código (por ejemplo, si se sube a GitHub) ya tiene esos datos. Los mensajes de error con demasiado detalle también le dan pistas a un atacante. Pertenece a MASVS-STORAGE y, en segundo plano, MASVS-CODE. Aparece en scripts/conexion.php, bd_php/conexion.php, scripts/login.php, scripts/add_comment.php, scripts/evento_detalle.php, scripts/registro_evento.php, scripts/editar_evento.php, paginas/login.php, paginas/registro.php, index.php, scripts/load_events.php, scripts/obtener_eventos.php, scripts/eventos.php, scripts/registro.php y bd_php/usuarioBD.php.

Busqué en el código la cadena new mysqli("localhost","root","",...) y aparece repetida en 14 archivos, con el usuario root y la contraseña vacía escritos directo. Además bd_php/usuarioBD.php tenía error_reporting(E_ALL) e ini_set("display_errors",1), y mostraba el mensaje de error de MySQL al usuario con die("Error de conexión: " . $e->getMessage()), lo que deja ver rutas, nombres de tablas y la versión de MySQL a cualquiera que visite la página. El proyecto tampoco tenía archivo .gitignore, así que si algún día se creaba un archivo de configuración con credenciales reales, se habría subido al repositorio sin ningún control.

Le puse probabilidad 2 (media) e impacto 2 (medio), así que el riesgo es Medio (4 de 9). Con root y sin contraseña quien llegue a la base de datos tiene control total, pero para eso primero necesita acceso al código o al servidor.

VULN-04 — Subida de archivos sin validar

Al crear un evento se puede subir una foto, pero la aplicación no comprueba que de verdad sea una imagen. Si alguien sube un archivo PHP, el servidor lo puede ejecutar como código, lo que se conoce como ejecución remota de código. Pertenece a MASVS-CODE y, en segundo plano, MASVS-PLATFORM. Está en scripts/registro_evento.php y scripts/editar_evento.php (campo "foto").

Con una cuenta normal, creada desde el registro público, creé un evento y en la foto subí un archivo shell.php que contiene <?php echo 'RCE_'.(6*7); ?>. La aplicación solo revisa $foto['error'] y mueve el archivo a la carpeta fotos/ con el mismo nombre y extensión que manda el navegador, sin comprobar qué contiene en realidad. Al abrir fotos/shell.php directamente, el servidor lo ejecutó como PHP y respondió RCE_42, o sea que cualquiera podría correr código en el servidor con los permisos del servidor web.

Le puse probabilidad 3 (alta) e impacto 3 (alto), así que el riesgo es Crítico (9 de 9). Con una cuenta normal se puede ejecutar código propio en el servidor.

Hallazgos y correcciones

Los seis hallazgos se corrigieron en v2-corregida. Esta tabla los resume, y debajo se explica cada corrección con el código de antes y de después.

Hallazgo	Área MASVS	Riesgo
VULN-01 — Inyección SQL	CODE	**Crítico**
VULN-02 — Control de acceso roto en las funciones de administrador	AUTH	**Crítico**
VULN-03 — Contraseñas guardadas con MD5	CRYPTO	**Medio**
VULN-04 — Subida de archivos sin validar	CODE	**Crítico**
VULN-05 — Credenciales escritas dentro del código	STORAGE	**Medio**
VULN-06 — Manejo inseguro de la sesión	AUTH	**Medio**

*Las claves, los hashes y las cookies de sesión están ocultos con **** en todas las capturas. En las capturas de código, las líneas resaltadas son las que importan para cada hallazgo, y el número de línea es el real del archivo.*

VULN-01 — Inyección SQL

Cambié todas las consultas que armaban el SQL con lo que escribe el usuario por consultas preparadas (prepare y bind_param de mysqli). En una consulta preparada, la instrucción SQL y los datos viajan por separado, así que lo que escriba el usuario nunca se interpreta como código. Además, los valores que deben ser números, como mes, pagina e id, ahora se convierten a entero antes de usarlos. El login quedó en una sola función, authenticate(), dentro de includes/auth.php.

En v1-original, scripts/login.php arma la consulta pegando directamente lo que escribe el usuario (líneas 21 a 23):

PHP login.php

v1-original › scripts › login.php

```
16  // Procesar formulario de login
17  if ($_SERVER["REQUEST_METHOD"] == "POST") {
18      $email = $_POST["email"];
19      $password = $_POST["pass"];
20
21      // Consultar usuario en la base de datos
22      $sql = "SELECT * FROM users WHERE email_us = '$email' AND password_us =
    '$password'";
23      $result = $conn->query($sql);
24
25      if ($result->num_rows > 0) {
26        // Usuario encontrado, iniciar sesión
27        session_start();
28        $_SESSION["logged_in"] = true;
29        $_SESSION["email"] = $email;
30        // Redirigir a la página de dashboard
31        header("Location: ../paginas/calendario.php");
32        exit;
33    } else {
```

v1-original PHP UTF-8 Espacios: 4

En v2-corregida, el login llama a la función authenticate() en lugar de armar el SQL a mano:

PHP login.php

v2-corregida › scripts › login.php

```php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $conn = db();
    $email = trim($_POST["email"] ?? '');
    $clave = $_POST["pass"] ?? '';

    $user = ($email !== '' && $clave !== '') ? authenticate($conn, $email, $clave) : null;

    if ($user) {
        login_user($user['email_us'], $user['Name_us']); // regenera el ID de sesion
        header("Location: ../paginas/calendario.php");
        exit;
    }
    http_response_code(401);
    header("Content-Type: application/json");
    echo json_encode(array("success" => false, "error" => "Credenciales incorrectas"));
    $conn->close();
    exit;
```

v2-corregida PHP UTF-8 Espacios: 4

Y esa función usa una consulta preparada, donde los datos viajan separados de la instrucción SQL (includes/auth.php):

PHP auth.php

v2-corregida › includes › auth.php

```php
function authenticate($conn, $correo, $clave) {
    $st = $conn->prepare('SELECT ID_us, Name_us, email_us, password_us FROM users WHERE email_us = ?');
    $st->bind_param('s', $correo);
    $st->execute();
    $fila = $st->get_result()->fetch_assoc();

```

v2-corregida PHP UTF-8 Espacios: 4

VULN-02 — Control de acceso roto

Creé includes/auth.php con las funciones require_login() y require_admin(). Los 14 scripts de administración ahora piden sesión iniciada y rol de administrador antes de leer o escribir cualquier dato. El rol (es_admin) se calcula en el servidor a partir de ADMIN_EMAIL, así que el usuario no puede cambiarlo desde el navegador.

En v1-original, eliminar_usuario.php solo inicia la sesión, pero no revisa quién es el usuario ni qué rol tiene, así que cualquiera que llegue al archivo puede borrar (líneas 28 a 35):

PHP eliminar_usuario.php

v1-original › scripts › eliminar_usuario.php

```
1  <?php
2  session_start();
3
4  include "conexion.php";
⋮
28 if(!empty($_POST)){
29     $id_usuario = $_POST['id'];
30     $id_usuario = mysqli_real_escape_string($conection, $id_usuario);
31
32     $query_delete = mysqli_query($conection, "DELETE FROM users WHERE ID_us =
   $id_usuario");
33
34     if($query_delete){
35         header("location: lista_usuarios.php");
```

v1-original PHP UTF-8 Espacios: 4

En v2-corregida, el archivo empieza llamando a require_admin():

PHP eliminar_usuario.php

v2-corregida › scripts › eliminar_usuario.php

```php
<?php
require_once __DIR__ . '/../includes/auth.php';
require_admin(); // VULN-02: solo administradores

include "conexion.php";

if(!empty($_POST)){
    $id_usuario = $_POST['id'];
    $id_usuario = (int)$id_usuario;

    $query_delete = mysqli_query($conection, "DELETE FROM users WHERE ID_us = $id_usuario");

    if($query_delete){
```

v2-corregida PHP UTF-8 Espacios: 4

Esa función exige que haya sesión iniciada y que el usuario sea administrador (includes/auth.php):

PHP auth.php

v2-corregida › includes › auth.php

```php
/** Exige sesión iniciada Y rol administrador (control de acceso en el servidor). */
function require_admin() {
    require_login();
    if (empty($_SESSION['es_admin'])) {
        http_response_code(403);
        exit('Acceso denegado');
    }
}
```

v2-corregida PHP UTF-8 Espacios: 4

VULN-03 — Contraseñas guardadas con MD5

Cambié md5() por password_hash($clave, PASSWORD_DEFAULT), que usa bcrypt (un método más lento y con sal), en todos los lugares donde se crea o se edita una contraseña. También agregué una migración: si una cuenta vieja todavía tiene

hash MD5, en su primer login correcto se valida una vez y se guarda de nuevo con bcrypt.

En v1-original la contraseña se guarda con md5() (línea 17):

PHP registro_usuario.php

v1-original › scripts › registro_usuario.php

```
15  $Name_us = $_POST['Name_us'];
16  $email_us  = $_POST['email_us'];
17  $password_us  = md5($_POST['password_us']);
 ⋮
27      $query_insert = mysqli_query($conection,"INSERT INTO
    users(Name_us,email_us,password_us)
28
    VALUES('$Name_us','$email_us','$password_us')");
```

v1-original PHP UTF-8 Espacios: 4

En v2-corregida se usa password_hash() (línea 18) y los datos van en una consulta preparada:

PHP registro_usuario.php

v2-corregida › scripts › registro_usuario.php

```
16  $Name_us = trim($_POST['Name_us']);
17  $email_us = trim($_POST['email_us']);
18  $password_us = password_hash($_POST['password_us'], PASSWORD_DEFAULT); //
    VULN-03: bcrypt con sal (antes md5)
 ⋮
27      $ins = $conection->prepare("INSERT INTO
    users(Name_us,email_us,password_us) VALUES(?,?,?)");
28      $ins->bind_param("sss", $Name_us, $email_us, $password_us);
```

v2-corregida PHP UTF-8 Espacios: 4

Además, al iniciar sesión, si la cuenta todavía tiene un hash MD5 viejo se acepta una sola vez y se cambia por bcrypt (includes/auth.php):

PHP auth.php

v2-corregida › includes › auth.php

```php
$hash = $fila['password_us'];
$ok = false;
$migrar = false;
if (password_verify($clave, $hash)) {
    $ok = true;
    $migrar = password_needs_rehash($hash, PASSWORD_DEFAULT);
} elseif (preg_match('/^[a-f0-9]{32}$/i', $hash) &&
hash_equals(strtolower($hash), md5($clave))) {
    $ok = true;
    $migrar = true;
}
```

v2-corregida PHP UTF-8 Espacios: 4

VULN-04 — Subida de archivos sin validar

Agregué includes/upload.php con la función guardar_foto_evento(). Revisa que el archivo venga de una subida real (is_uploaded_file), limita el tamaño, comprueba el tipo real con fileinfo en lugar del que dice el navegador y lo compara con una lista de imágenes permitidas. Después confirma con getimagesize() que de verdad sea una imagen y la guarda con un nombre aleatorio y la extensión que corresponde a su tipo, nunca la que mandó el cliente. Como protección extra puse un fotos/.htaccess que apaga la ejecución de PHP en esa carpeta.

En v1-original, registro_evento.php guarda la foto con el nombre que manda el navegador y sin revisar qué contiene (líneas 45 a 48):

PHP registro_evento.php

v1-original › scripts › registro_evento.php

```
40  if ($foto['error'] !== 0) {
41    echo "Error: Foto inválida";
42    exit;
43  }
44
45  // Mover la foto a una carpeta temporal
46  $rutaTemporal = $foto['tmp_name'];
47  $rutaDestino = '../fotos/' . $foto['name'];
48  move_uploaded_file($rutaTemporal, $rutaDestino);
```

v1-original PHP UTF-8 Espacios: 4

En v2-corregida, includes/upload.php revisa que sea una subida real, que no pese más de 2 MB, que su tipo real sea de imagen y que se pueda leer como imagen. Después la guarda con un nombre aleatorio:

PHP upload.php

v2-corregida › includes › upload.php

```php
function guardar_foto_evento($f) {
    if (!is_array($f) || ($f['error'] ?? UPLOAD_ERR_NO_FILE) !== UPLOAD_ERR_OK) {
        return null;
    }
    if (!is_uploaded_file($f['tmp_name']) || $f['size'] > 2 * 1024 * 1024) {
        return null;
    }
    $permitidos = ['image/jpeg' => 'jpg', 'image/png' => 'png', 'image/gif' => 'gif', 'image/webp' => 'webp'];
    $mime = (new finfo(FILEINFO_MIME_TYPE))->file($f['tmp_name']);
    if (!isset($permitidos[$mime]) || @getimagesize($f['tmp_name']) === false) {
        return null;
    }
    $nombre = bin2hex(random_bytes(16)) . '.' . $permitidos[$mime];
    if (!move_uploaded_file($f['tmp_name'], __DIR__ . '/../fotos/' . $nombre)) {
        return null;
    }
    return '../fotos/' . $nombre;
}
```

v2-corregida PHP UTF-8 Espacios: 4

Además, la carpeta fotos/ tiene un archivo .htaccess que impide ejecutar PHP ahí:

CFG .htaccess

v2-corregida › fotos › .htaccess

```
<FilesMatch "\.(php|phtml|phar|php[0-9])$">
    Require all denied
</FilesMatch>
php_flag engine off
Options -ExecCGI -Indexes
```

v2-corregida Apache UTF-8 Espacios: 4

VULN-05 — Credenciales escritas en el código

Dejé la conexión en includes/db.php e includes/config.php, que leen el host, el usuario, la clave y la base de datos de variables de entorno o del archivo includes/config.local.php, que no se sube al repositorio. Los errores de conexión y de consulta ahora se guardan con error_log() y al usuario solo le sale un mensaje general. También agregué un .gitignore que excluye config.local.php, los archivos *.env y los registros temporales de las pruebas.

En v1-original, scripts/conexion.php trae el usuario root y la contraseña vacía escritos directo (líneas 3 a 6):

PHP conexion.php

v1-original › scripts › conexion.php

```
1  <?php
2
3      $host = 'localhost';
4      $user = 'root';
5      $password = '';
6      $db = 'totoninin';
7
8      $conection = @mysqli_connect($host,$user,$password,$db);
9
10     if(!$conection){
11         echo "Error en la conexión";
12     }
```

v1-original PHP UTF-8 Espacios: 4

En v2-corregida, conexion.php ya no tiene datos de acceso y le pide la conexión a db():

PHP conexion.php

v2-corregida › scripts › conexion.php

```
1  <?php
2  // VULN-05: las credenciales ya no estan en el codigo (ver
   includes/config.php)
3  require_once __DIR__ . '/../includes/db.php';
4  $conection = db();
```

v2-corregida PHP UTF-8 Espacios: 4

Los datos se leen de variables de entorno o de config.local.php, que no se sube a Git (includes/config.php):

PHP config.php

v2-corregida › includes › config.php

```php
function cfg($clave, $defecto = null) {
    static $local = null;
    if ($local === null) {
        $archivo = __DIR__ . '/config.local.php';
        $local = is_file($archivo) ? (require $archivo) : [];
    }
    $valor = getenv($clave);
    if ($valor !== false && $valor !== '') {
        return $valor;
    }
    return $local[$clave] ?? $defecto;
}
```

v2-corregida PHP UTF-8 Espacios: 4

VULN-06 — Manejo inseguro de la sesión

Creé includes/session.php. La cookie de sesión ahora lleva HttpOnly (el código de la página no puede leerla) y SameSite=Lax (no se manda desde otros sitios), y Secure cuando se usa HTTPS; la sesión expira a los 30 minutos sin actividad; se llama a session_regenerate_id(true) en cada login; y logout_user() vacía $_SESSION, destruye la sesión en el servidor y expira la cookie en el navegador. El login ahora siempre responde "Credenciales incorrectas" sin decir cuál fue el motivo, y el rol es_admin se calcula una sola vez en el servidor al autenticar.

En v1-original, el login inicia la sesión sin cuidar la cookie ni cambiar el ID de sesión (líneas 27 a 29):

PHP login.php

v1-original › scripts › login.php

```
25  if ($result->num_rows > 0) {
26    // Usuario encontrado, iniciar sesión
27    session_start();
28    $_SESSION["logged_in"] = true;
29    $_SESSION["email"] = $email;
30    // Redirigir a la página de dashboard
31    header("Location: ../paginas/calendario.php");
32    exit;
```

v1-original PHP UTF-8 Espacios: 4

En v2-corregida, includes/session.php configura la cookie con HttpOnly y SameSite (líneas 14 a 19), y login_user() cambia el ID de sesión (línea 41):

PHP session.php

v2-corregida › includes › session.php

```
11      $https = !empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off';
12      ini_set('session.use_strict_mode', '1');   // rechaza IDs de sesión no
    emitidos por el servidor
13      ini_set('session.use_only_cookies', '1');
14      session_set_cookie_params([
15          'lifetime' => 0,
16          'path'     => '/',
17          'secure'   => $https,
18          'httponly' => true,
19          'samesite' => 'Lax',
20      ]);
21      session_start();
 :
38  /** Marca la sesión como autenticada y renueva el ID (evita fijación de
    sesión). */
39  function login_user($correo, $nombre) {
40      secure_session_start();
41      session_regenerate_id(true);
42      $admin = cfg('ADMIN_EMAIL', '');
43      $_SESSION['correo'] = $correo;
44      $_SESSION['nombre'] = $nombre;
45      $_SESSION['es_admin'] = ($admin !== '' && strcasecmp($correo, $admin) ===
    0);
46      $_SESSION['ultimo_acceso'] = time();
47  }
```

v2-corregida PHP UTF-8 Espacios: 4

Verificación e informe

Procedimiento de prueba

Para cada hallazgo mandé a la aplicación las peticiones de ataque (por ejemplo, el texto x' OR '1'='1 en el login, un POST sin sesión a las páginas de administrador o un archivo shell.php como foto de un evento) y revisé la respuesta, primero en v1-original y después en v2-corregida. También comprobé que las funciones normales siguieran funcionando: iniciar sesión con un usuario válido, ver el calendario, usar el panel de administración con la cuenta de administrador, registrar un usuario y abrir la página de inicio.

En total fueron 24 pruebas, 19 de seguridad y 5 de funcionamiento normal. En v1-original fallaron 16 de las 19 pruebas de seguridad, y en v2-corregida pasaron las 24.

Resultados de las pruebas

VULN-01 · Inyección SQL

Resultado del ataque antes de corregir:

```
RESULTADO DE LA PRUEBA
VULN-01 · antes de corregir

Petición:  POST scripts/login.php
email =  x' OR '1'='1
pass  =  x' OR '1'='1
Respuesta:  302 Found, Location: ../paginas/calendario.php
Set-Cookie: PHPSESSID=****
✘ FALLA: entró sin credenciales válidas.

Pruebas                                          NO CUMPLE
```

Resultado del mismo ataque después de corregir:

```
RESULTADO DE LA PRUEBA
VULN-01 · después de corregir

Petición:  POST scripts/login.php
email =  x' OR '1'='1
pass  =  x' OR '1'='1
Respuesta:  401 Unauthorized
{"success":false,"error":"Credenciales incorrectas"}
✔ OK: el texto se trata como texto normal y no se inicia sesión.

Pruebas                                                    CUMPLE
```

VULN-02 · Control de acceso roto

Resultado de la prueba sin iniciar sesión, antes de corregir:

```
RESULTADO DE LA PRUEBA
VULN-02 · antes de corregir

Petición:  POST scripts/eliminar_usuario.php?id=3 (sin iniciar sesión)
Respuesta:  302 Found, Location: lista_usuarios.php
✘ FALLA: el usuario con id=3 se eliminó sin que nadie hubiera iniciado sesión.

Pruebas                                                    NO CUMPLE
```

Resultado de la misma prueba después de corregir:

```
RESULTADO DE LA PRUEBA
VULN-02 · después de corregir

Petición:  POST scripts/eliminar_usuario.php?id=3 (sin iniciar sesión)
Respuesta:  302 Found, Location: ../paginas/login.php
✔ OK: lo manda al login y el usuario sigue en la base de datos.

Pruebas                                                    CUMPLE
```

VULN-03 · Contraseñas guardadas con MD5

Lo que se guardaba en la base de datos antes de corregir:

```
RESULTADO DE LA PRUEBA
VULN-03 · antes de corregir

Consulta:  SELECT password_us FROM users WHERE email_us = 'nuevo@totoninin.test'
Resultado:  [hash MD5 de 32 caracteres, oculto]
✘ FALLA: MD5 sin sal, se puede descifrar con un diccionario en segundos.

Pruebas                                                    NO CUMPLE
```

Lo que se guarda después de corregir:

```
RESULTADO DE LA PRUEBA
VULN-03 · después de corregir

Consulta:  SELECT password_us FROM users WHERE email_us = 'nuevo@totoninin.test'
Resultado:  [hash bcrypt $2y$10$..., oculto]
✔ OK: bcrypt usa sal y es lento de adivinar, no se descifra con un diccionario.

Pruebas                                                    CUMPLE
```

VULN-04 · Subida de archivos sin validar

Resultado de subir shell.php antes de corregir:

```
RESULTADO DE LA PRUEBA
VULN-04 · antes de corregir

Petición 1:  POST registro_evento.php, foto = shell.php (tipo declarado:
image/jpeg)
Petición 2:  GET fotos/shell.php
Respuesta:  RCE_42
✘ FALLA: el archivo .php subido se ejecutó en el servidor.

Pruebas                                                    NO CUMPLE
```

Resultado de subir shell.php después de corregir:

```
RESULTADO DE LA PRUEBA
VULN-04 · después de corregir

Petición:  POST registro_evento.php, foto = shell.php (tipo declarado: image/jpeg)
Respuesta:  400 Bad Request, "Error: archivo de imagen no válido"
✔ OK: se rechaza porque su contenido real no es una imagen y no se guarda.

Pruebas                                                    CUMPLE
```

VULN-05 · Credenciales escritas en el código

Resultado de buscar las credenciales en el código antes de corregir:

```
RESULTADO DE LA PRUEBA
VULN-05 · antes de corregir

Búsqueda en el código:  new mysqli("localhost", "root"
Archivos con coincidencias:  14
✘ FALLA: usuario root y clave vacía escritos en 14 archivos.

Pruebas                                                    NO CUMPLE
```

Resultado de la misma búsqueda después de corregir:

```
RESULTADO DE LA PRUEBA
VULN-05 · después de corregir

Búsqueda en el código:  new mysqli("localhost", "root"
Archivos con coincidencias:  0
.gitignore incluye:  includes/config.local.php
✔ OK: no quedan credenciales en el código y el archivo con secretos no se sube a
Git.

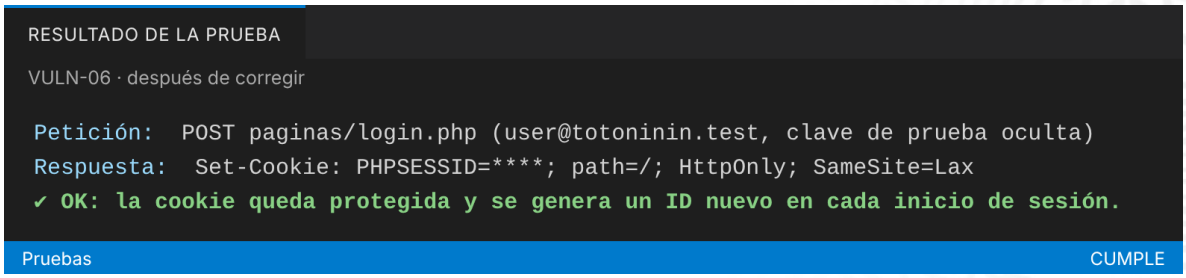
Pruebas                                                    CUMPLE
```

VULN-06 · Manejo inseguro de la sesión

Resultado del inicio de sesión antes de corregir:



Resultado del inicio de sesión después de corregir:



Conclusiones de la repetición de pruebas

Al repetir las pruebas con las correcciones aplicadas, el intento de inyección SQL en el login fue bloqueado y el intento de subir un archivo PHP disfrazado de imagen fue rechazado. El acceso a las páginas de administración sin sesión redirige al login, la cookie de sesión ya lleva HttpOnly y SameSite, y las contraseñas nuevas se guardan con bcrypt. Las funciones normales de la plataforma (ver el calendario, registrar usuarios, el panel de administración) siguen funcionando sin problemas.

Con esto se concluye que TotoNinin corrigió los 6 hallazgos, incluidos los 3 de riesgo Crítico (inyección SQL, control de acceso roto y subida de archivos sin validar). Quedan pendientes algunas mejoras que no entraron en esta auditoría: activar HTTPS en el servidor de producción, poner un límite de intentos de inicio de sesión, agregar un aviso de privacidad con la opción de eliminar la cuenta, y usar en

producción un usuario de MySQL con permisos mínimos y contraseña fuerte, nunca root sin clave.

Las dos versiones del código, v1-original y v2-corregida, están en el repositorio de GitHub https://github.com/hectorjua28/Repositorio-Pruebas-Totonin.